

NUST

NATIONAL UNIVERSITY
OF SCIENCES & TECHNOLOGY

Faculty of Computing

CS 417: Large Language Models (LLMs)

Assignment-01

BSCS-13

LLM-Group-02

Name	Qalam ID
Rameen Arshad	461136
Laiba Riaz	464486
Adeena Reeham	480841
Zaynab Shahid	458224

Instructor: Prof. Dr. Naima Iltaf

Building an Urdu BPE Tokenizer from Scratch

CS 417 Large Language Models, Assignment 1. LLM-Group-02: Rameen Arshad, Laiba Riaz, Adeena Reeham, Zaynab Shahid. October 2026.

1. Corpus and normalisation

We collected 57.05 MB ourselves through the MediaWiki API (Urdu Wikipedia, encyclopedia, and Urdu Wikisource, literature), downloaded the official Urdu Wikipedia XML dump of 1 October 2026 (skipping pages already collected through the API, and setting aside 434,745 articles under 1 KB, which are mostly bot-generated one-line stubs), and added the published UFAL UrMonoCorp (mixed web text from 2014, CC BY-NC-SA 3.0). After cleaning, the corpus is **1,210.14 MB, 339,466 documents and 147.83 million words** (1,420,018 distinct), split 98/1/1 by document with seed 1337, separately inside each source: train 1,185.94 MB, validation 11.93 MB, test 12.27 MB. UFAL is 58.9% of the bytes (94.0% before we added the dump) and Wikipedia 41.0%. UFAL's sentences are shuffled (neighbouring lines share 2.8% of their words, against 11.1% in Wikipedia dump articles and 9.9% in those from the API), so its documents are groups of 40 unrelated sentences. Cleaning took the raw 1,411.97 MB down to 1,210.14 MB, a net 201.83 MB (14.3%) after the document separators are added, mostly wiki markup and reference sections; every step and its effect are in DATA_CARD.md.

Urdu text mixes code points that look identical: Arabic yeh and alef maksura for Urdu yeh (603,854 found), Arabic kaf for Urdu kaf (113,402), Arabic heh for heh goal (133,637), plus tatweel, invisible marks and three digit sets. Our `normalize()` maps them to one form, keeps U+06BE apart from U+06C1, converts all digits to ASCII (99.9% already were), keeps diacritics (NFC, repeats collapsed) and keeps ZWNJ only between two joining letters, where Urdu uses it on purpose (3,963 of 7,271 kept; every one sits between two letters in the final files). On 100 MB of UFAL text it removes 5.6% of distinct words. The corpus is normalised once, before training; the tokenizer never calls `normalize`, so `decode(encode(x)) == x` holds for any string.

2. The tokenizer

Specification. IDs 0 to 255 are the bytes, 256 is `<|endoftext|>` (never split, never merged: training cuts the text at it), and the 49,743 merges take IDs 257 to 49,999 in the order learned. Ties go to the greater pair compared as byte strings, not as IDs (ID 257 = `ab` is smaller than ID 98 = `b`). Encoding applies merges in the order learned; decoding joins all bytes and decodes once, because one token can hold half a letter.

Pre-tokenisation. GPT-2's pattern uses `\p{L}+` for words. Urdu diacritics are combining marks (category M) and ZWNJ is a format character (Cf), so GPT-2 sends both to its punctuation branch: اُردُو becomes five pieces, ا ُ ر د ُ و, and no merge can ever span a mark. In train.txt this would cut 1,389,662 Urdu word occurrences that carry a mark (0.99% of Urdu words), including the very common اُس, اُن and تعالیٰ. Our pattern:

```
?[\p{L}\p{M}\u200c]+| ?\p{N}| ?[^\s\p{L}\p{M}\p{N}\u200c]+|\s+(?!\S)|\s+
```

The first alternative is a word: one optional space followed by a run of letters (`\p{L}`), combining marks (`\p{M}`) and ZWNJ, so اُردُو, تعالیٰ and بین‌الاقوامی each stay one piece and the space before a word joins it, as in GPT-2. Each digit (`\p{N}`) is its own piece, as in LLaMA, so a number is always cut the same way (1947 is never 194 + 7), at the cost measured in the table below; runs of other symbols also form their own pieces, with the same optional leading space; keeping them apart means ہے and ہے۔, or سال and سال1947, share one token instead of learning every glued variant. The last two alternatives take whitespace the GPT-2 way: a run of spaces gives up its final space to the next word, and newlines and tabs are never attached to a word; every character belongs to exactly one class, so the pieces always join back to the input.

Number rule	test.txt tokens	Tokens on numbers
one digit per piece (final)	1,864,014	106,164
one to three digits (variant)	1,814,905	54,761

Splitting every digit costs 2.71% more tokens on test.txt (49,109 tokens; numbers take 1.94 times as many tokens) and frees 1,822 vocabulary slots that the variant spends on multi-digit numbers.

Training. The reference trainer recounts every pair of every word occurrence after each merge, which costs about (merges x corpus size). The fast trainer (i) counts each distinct word once, weighted by its frequency: the 172,744,423 pre-tokens of train.txt collapse to 1,387,085 distinct ones; (ii) keeps an index from each pair to the words containing it, so a merge only touches those words: it subtracts their old pairs and adds their new ones; (iii) at encoding time, caches the IDs of every word already encoded, so almost every word is a dictionary lookup. The best pair comes from a binary heap with lazy deletion: when a count changes we push a new entry and skip any popped entry whose count is out of date. Python's `heapq` is a min-heap and bytes cannot be negated, so each entry is (−count, inv(bytes of a), inv(bytes of b), −a, −b), where inv maps every byte x to 255 − x and appends 256; this reverses byte-string order, prefixes included, so the smallest entry is the highest count with the greatest byte pair, and the IDs settle the rare tie between two different tokens with the same bytes (the reference trainer uses the same order).

Measure	Value
Training time (fast, 1,186 MB of train.txt)	0.20 h (723 s)
Merges learned	49,743
Peak memory	1,668 MB
Machine	11th Gen Intel(R) Core(TM) i5-1145G7 @ 2.60GHz, 8 logical CPUs, 15.7 GB RAM, Windows-11-10.0.26200-SP0, Python 3.12.10
Fast vs reference, 1 MB sample	500 merges, identical: yes; reference 369.2 s, fast 4.3 s (86x)
train.bin	181,315,074 tokens (363 MB), 1.25 tokens/word, 6.54 bytes/token, encoded in 391 s

Measure	Value
val.bin	1,824,155 tokens (4 MB), 1.25 tokens/word, 6.54 bytes/token, encoded in 4 s
check_bins.py	passed. train: max ID 49999, 202 documents round-tripped; val: max ID 49997, 3,394 documents round-tripped

Tests (`tests/`) check the hand-derived sanity corpus of Sennrich et al. (the handout's own example was missing from our copy), the byte-versus-ID tie-break, overlapping pairs (`aaa`), losslessness on emoji, English, mixed text, newlines, the empty string and 300 random Unicode strings, and that the fast trainer matches the reference on 200 random corpora and on the 1 MB sample.

3. What did the merges learn?

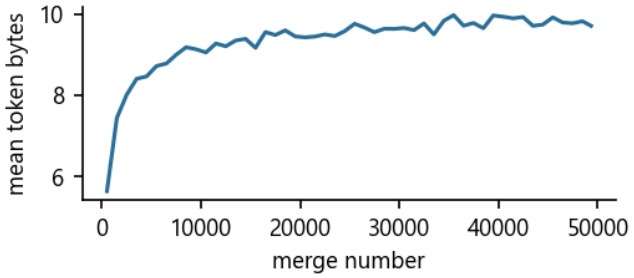
Of the first 50 merges, 35 rebuild a two-byte letter (merges 1, 2, 5, 6, 7, 8, 9, 11, 13, 14, 15, 16...) and 4 produce a token that is not valid UTF-8 on its own (merges 3, 4, 12, 35), typically a space joined to the first byte of a letter; 3 attach a leading space to a letter. The first whole word is merge 32, کی_. Over all 49,743 merges, 101 tokens (0.20%) are not valid UTF-8 alone. The longest token has 35 bytes.

Merges 1 to 50 (merge *k* creates ID 256 + *k*; _ is a leading space; `letter`: rebuilds a two-byte letter; `space + letter`: adds a space to one; `partial`: not valid UTF-8 alone; `word`: a whole word; `N letters`: a longer piece):

Merge	Token	Bytes	Note	Merge	Token	Bytes	Note	Merge	Token	Bytes	Note
1	ا	2	letter	18	ت	2	letter	35	<20 d8 a7 d8>	4	partial
2	ی	2	letter	19	ہ_	3	space + letter	36	ج_	3	letter
3	<20 d8>	2	partial	20	ھ	2	letter	37	ان	4	2 letters
4	<20 d9>	2	partial	21	یں	4	2 letters	38	میں_	7	word
5	ک	2	letter	22	ب_	3	letter	39	ار	4	2 letters
6	ر	2	letter	23	س_	3	letter	40	د_	3	letter
7	و	2	letter	24	س	2	letter	41	ہے_	5	word
8	ہ	2	letter	25	د	2	letter	42	ٹ	2	letter
9	ے	2	letter	26	گ	2	letter	43	و_	3	letter
10	ک_	3	space + letter	27	ت_	3	letter	44	ق	2	letter
11	ن	2	letter	28	ب	2	letter	45	ف	2	letter
12	<d8 a7 d8>	3	partial	29	ن_	3	letter	46	چ	2	letter
13	ل	2	letter	30	ور	4	2 letters	47	ع	2	letter
14	م	2	letter	31	۔	2	letter	48	ل_	3	letter
15	ں	2	letter	32	کی_	5	word	49	ج	2	letter
16	ہ_	3	letter	33	پ_	3	letter	50	پ	2	letter
17	ا_	3	space + letter	34	کے_	5	word				

Every 5,000th merge:

Merge	ID	Token	Bytes	Note
5000	5256	ڈوم_	7	3 letters
10000	10256	اطین	8	4 letters
15000	15256	لفٹ_	7	3 letters
20000	20256	روزہ	8	4 letters
25000	25256	گرائے_	11	5 letters
30000	30256	جیون	8	4 letters
35000	35256	ہاروے_	11	5 letters
40000	40256	wick	4	4 letters
45000	45256	ویلہ	8	4 letters
49743	49999	چیلنجز_	13	6 letters



The 20 longest tokens:

Merge	Token	Bytes	Characters
48560	کیرولائناٹاؤنمیئر_	35	18
35761	ٹیمکھیلے جیتے ہارے	32	16
48945	لیسٹرشائرکاؤنٹی	30	15
12266	انسائیکلوپیڈیا_	29	15
28875	الاقوامیٹوننٹی_	29	15
28972	انٹرکانٹینینٹل_	29	15
46355	وانڈررزاسٹیڈیم_	29	15
44360	انسائیکلوپیڈیا	28	14
47871	ہیمپشائرکاؤنٹی	28	14
14226	انانکارپوریٹڈ_	27	14
37099	کیرولائناٹاؤن_	27	14
33556	کھیلے جیتے ہارے	26	13
12987	ایسٹروفیزیکل_	25	13
21009	خیبرپختونخوا_	25	13
31710	نارتھمبرلینڈ_	25	13
33852	امیرالمومنین_	25	13
39008	کوئنزاسپورٹس_	25	13
42770	ہارٹفورڈشائر_	25	13
42813	طالبانائزیشن_	25	13
44632	امیرالمؤمنین_	25	13

Every Urdu letter is two bytes and almost every word starts with a space, so the earliest merges rebuild letters and attach the space to the first byte of a word; those partial tokens are still used by encode wherever no longer merge applies. Later merges join letters into suffixes and frequent function words, then into whole words, and token length levels off near 10 bytes (about five letters) after

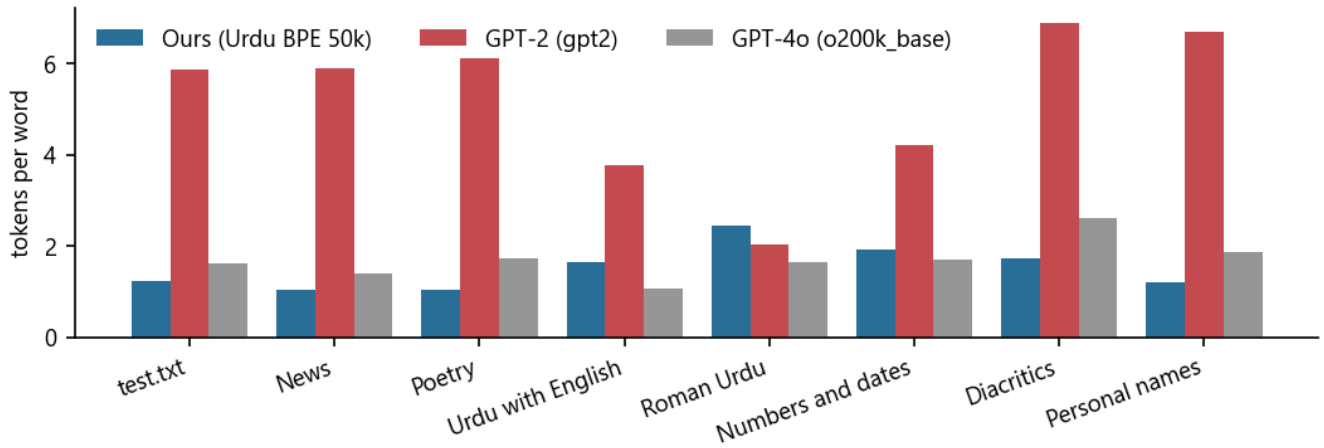
15,000 merges, where the remaining merges are mostly rarer words, names and word pieces. The longest tokens are long names and loanwords (انسائیکلوپیڈیا, خیبرپختونخوا), but also a cleaning artifact: several are words glued together, such as ٹیمکھیلےجیتےہارے ("team played won lost") and لیسٹرشائرکاؤنٹی ("Leicestershire County"). They come from the header rows of cricket tables in the Wikipedia dump, whose cells lost their separators when the wikitext was converted to plain text (mwparserfromhell's `strip_code` in `scripts/common.py`): the glued header occurs 221 times in train.txt against 77 times with spaces, and "Leicestershire County" 164 times glued against once spaced. Putting a space between table cells during that conversion would remove them.

4. How does it compare with existing tokenizers?

Tokenizer	Vocabulary	Tokens	Tokens per word	Bytes per token
Ours (Urdu BPE 50k)	50,000	1,864,014	1.25	6.56
GPT-2 (gpt2)	50,257	8,763,431	5.86	1.39
GPT-4o (o200k_base)	200,019	2,445,261	1.64	5.00

test.txt: 3,394 documents, 1,495,267 words, 12.22 MB.

On test.txt our tokenizer needs 1.25 tokens per word, against 5.86 for GPT-2 (4.7 times as many), which falls back to raw bytes for most Urdu letters. o200k_base needs 1.64 (1.31 times ours): a 50k vocabulary spent on one language beats a 200k vocabulary shared by many. Among the hand-picked inputs, o200k_base does better on Urdu with English and Roman Urdu: text that our Urdu-only corpus rarely contains. It also does better on numbers and dates, because our pattern gives every digit its own token (section 2).



Tokens per word on the seven hand-picked inputs (normalised as the corpus is):

Input	Words	Ours (Urdu BPE 50k)	GPT-2 (gpt2)	GPT-4o (o200k_base)
News	40	1.05	5.90	1.40
Poetry	18	1.06	6.11	1.72
Urdu with English	23	1.65	3.78	1.09
Roman Urdu	20	2.45	2.05	1.65
Numbers and dates	34	1.94	4.21	1.71
Diacritics	18	1.72	6.89	2.61
Personal names	24	1.21	6.71	1.88

How the tokenizers cut a few words:

Word	Ours (Urdu BPE 50k)	GPT-2 (gpt2)	GPT-4o (o200k_base)
اُردُو	اُ رد وٗ	ا <d9> <8f> ر د <d9> <8f> و	ا رد ٗ و
پاکستان	پاکستان	<d9> <be> ا <da> <a9> س ت ا ن	پاکستان
خواہشیں	خواہ شیں	<d8> <ae> و ا <db> <81> <d8> <b4> <db> <8c> <da> <ba>	خوا ہ ش یں
یوسفزئی	یوسف زئی	<db> <8c> و س <d9> <81> <d8> <b2> <d8> <a6> <db> <8c>	ی وسف ز ئی
1947	1 9 4 7	19 47	194 7

5. Three inputs our tokenizer handles badly

1. URL and code. `https://www.example.com/news/2026/10/urdu?id=4471&lang=ur`

Tokenizer	Tokens	Bytes per token
Ours (Urdu BPE 50k)	38	1.5
GPT-2 (gpt2)	26	2.19
GPT-4o (o200k_base)	21	2.71

First tokens of ours, in order (▁ is a space): `ht` `t` `ps` `:` `//` `ww` `w` `.` `ex` `ample` `.` `com` `/` `new`

URLs were scrubbed from the corpus as personal data, so their pieces are rare. Our tokenizer gets 1.50 bytes per token here (test.txt averages 6.56).

2. Unnormalised Arabic letters. كتاب میں یہ لکھا ہے کہ ہم ایك ہیں

Tokenizer	Tokens	Bytes per token
Ours (Urdu BPE 50k)	30	1.93
GPT-2 (gpt2)	43	1.35
GPT-4o (o200k_base)	16	3.62

First tokens of ours, in order (▁ is a space): `<d9>` `<83>` تاب مے `<d9>` `<8a>` ں `<20 d9>` `<8a>` ہ لے `<d9>` `<83>` ها

Typed with Arabic kaf, yeh and heh (U+0643, U+064A, U+0647). normalize() maps them to the Urdu letters, so they almost never occur in train.txt and the tokenizer falls back to two bytes per letter. Normalising the input first fixes it. Our tokenizer gets 1.93 bytes per token here (test.txt averages 6.56).

3. Urdu digits. اگست ۱۹۴۷ اور ۲۰۲۶ کی قیمت ۱۲۵۰ روپے ۱۴

Tokenizer	Tokens	Bytes per token
Ours (Urdu BPE 50k)	36	1.94
GPT-2 (gpt2)	59	1.19
GPT-4o (o200k_base)	19	3.68

First tokens of ours, in order (▁ is a space): `<db>` `<b1>` `<db>` `<b4>` اگستں ▁ `<db>` `<b1>` `<db>` `<b9>` `<db>` `<b4>` `<db>` `<b7>`

Extended Arabic-Indic digits were converted to ASCII during cleaning, so every Urdu digit costs two byte tokens. Our tokenizer gets 1.94 bytes per token here (test.txt averages 6.56).

All candidates we measured:

Input	Characters	Ours (Urdu BPE 50k) tokens	GPT-2 (gpt2) tokens	GPT-4o (o200k_base) tokens
URL and code	57	38	26	21
Unnormalised Arabic letters	33	30	43	16
Urdu digits	39	36	59	19
Roman Urdu	78	35	30	24
Social media	56	37	63	25
Hindi (Devanagari)	43	35	69	14
Missing spaces	24	10	40	13

What normalisation is worth in tokens. The cases of `normalization_cases.py`, raw and after `normalize()`, where our token count changes:

Spelling problem	Ours raw	Ours normalised	Change	o200k raw	o200k normalised
ZWJ and direction marks	8	2	-75%	6	3
Arabic presentation forms	16	4	-75%	13	5
Tatweel stretching	7	2	-71%	4	3
Non-breaking spaces and runs of spaces	12	4	-67%	10	6
Arabic yeh in place of Urdu yeh	8	3	-62%	4	3
Arabic heh in place of heh goal	8	3	-62%	6	7
Alef maksura at the end of a word	7	3	-57%	7	6
Arabic kaf in place of Urdu kaf	7	3	-57%	4	5

The 8 largest savings are shown. On all 16 of 21 cases where it matters, normalising first cuts our tokens from 153 to 74 (-52%); the largest saving is "ZWJ and direction marks" (8 to 2 tokens). Because train.txt was normalised, the raw forms are all but absent from it and fall back to bytes or rare pieces; this is why the next assignment calls normalize() before encode() instead of building normalisation into the lossless tokenizer.

Choices and known limits. English and Roman Urdu were left out of the corpus on purpose (only natural code-mixing was kept), so they cost more tokens than in o200k_base; if the next model must handle them, 2 to 5% of such text would fix it. Numbers also cost more than in o200k_base, because we split every digit (section 2). Text typed with Arabic letters or Urdu digits is costly only because it is unnormalised: in the next assignment we run `normalize()` before `encode()`, which keeps the tokenizer lossless and makes those inputs as cheap as normal Urdu. Some slots go to spelling variants that are both in use (265 امیرالمومنین times and 159 امیرالمؤمنین in train.txt, both tokens), which we did not force together because both spellings are correct, and a few to junk such as laughter (ہاہاہا, 2,105 times). Adding the October 2026 Wikipedia dump took UFAL from 94% to 59% of train.txt: two junk tokens of our first vocabulary (a seven-syllable laughter and the footer میرےدیگرمضامین, "my other articles", with its spaces missing) dropped out, tokens per word fell 1.9% on the Wikipedia part of test.txt and 6% on our diacritics input, and rose only 0.5% on the UFAL part. The vocabulary still holds 2014 topics (طالبانائزیشن), and recent news and conversational Urdu remain the largest gap.

6. Code and data

- **Code and saved tokenizer:** https://github.com/reeham4411/Urdu-BPE-Tokenizer
- **Data (train/val/test.txt, train.bin, val.bin):** https://drive.google.com/drive/folders/15mM4_TXFahkNFSQh1qgycO18gAWOkmXo